

Spaceship Titanic

Project Report



Presented by

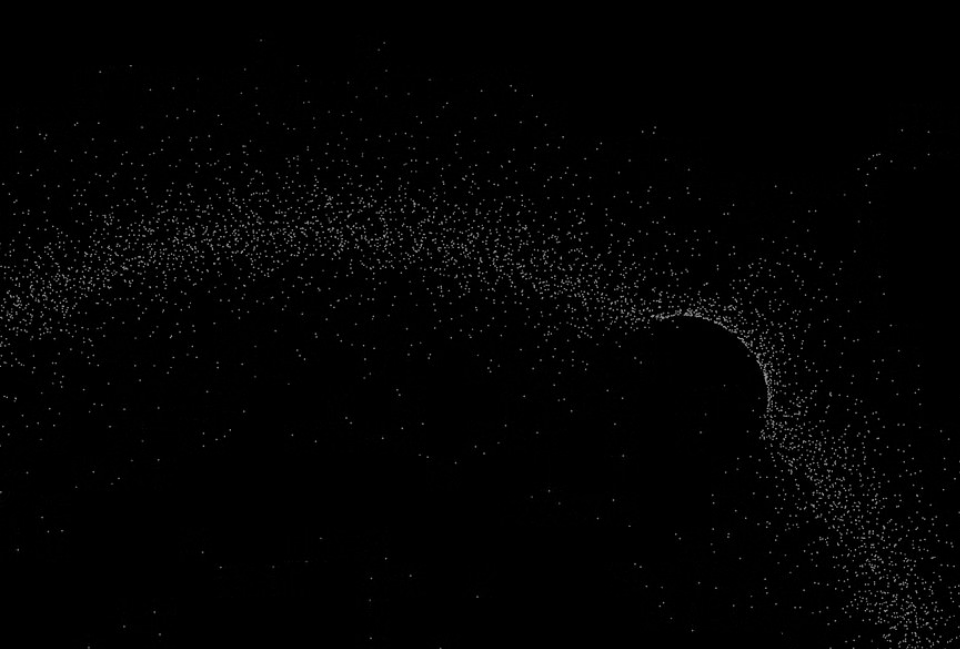
Arman Mahmoodzadeh

Daniel Herrera

Jose Mateo Ayala

Leonardo Granados-Castillo

Content



1 Team Responsibilities



2 Project Description



3 Dataset Description



4 Tools and Environment



5 Methods, Algorithms, Model
Performance Comparison



6 Final Results & Conclusions



Team Responsibilities

Arman

Team Lead

Arman and Daniel

Pair A – Logistic Regression Team

Responsibilities:

- Conduct preprocessing and encode categorical variables
- Scale numerical features
- Build baseline Logistic Regression model
- Tune hyperparameters
- Analyze model coefficients
- Generate ROC curve and AUC score

Leo and Jose

Pair B – Decision Tree Team

Responsibilities:

- Prepare tree-friendly data (label encoding, missing-value strategies)
- Train and tune Decision Tree models of different depths
- Visualize tree structure
- Analyze patterns in top-level splits
- Evaluate performance with accuracy, ROC, and AUC

All Four Members – Random Forest Model

Responsibilities:

- Integrate best feature-engineered datasets from both teams
- Train and tune Random Forest ensemble
- Conduct hyperparameter search (n_estimators, max_depth, etc.)
- Analyze feature importance
- Compare performance with baseline models
- Generate final predictions and interpret results

Tools & Environment



All analysis and development were completed using:

- **Python 3.x**
- **Jupyter Notebook**
- **NumPy**
- **pandas**
- **matplotlib & seaborn**
- **scikit-learn (sklearn)**
- **Graphviz** (Decision Tree visualization)

Project Description

Our project focuses on predicting whether the passengers on the *Spaceship Titanic* were transported to another dimension after a spacetime anomaly. Using the Kaggle dataset, our goal is to build a machine-learning model that predicts the “transported” status of each passenger based on various data. We will clean and prepare the data, create useful features, and test different machine-learning models. Overall, our goal is to build an accurate prediction model that can be used reliably. Our goal has been to clean and explore data, understand features, missing values, distributions, and correlations. We wanted to transform raw features into ones that are more useful for modelling and build one or more models. Then, evaluate the model's performance, use it to generate predictions, and learn from the results.

Each model was trained, tuned, and evaluated, and the final results were compared using accuracy, ROC curves, and AUC metrics. The project concludes by selecting the best-performing model for deployment.

Dataset

The Spaceship Titanic dataset is provided by **Kaggle** and includes demographic, travel, and spending information for passengers aboard the ship.

Train Dataset

Contains all features plus the target variable **Transported**.

Test Dataset

Contains all features minus the target label; used to generate model predictions.

Target Variable

- **Transported**
 - Boolean value
 - True → Passenger was transported to another dimension
 - False → Passenger remained in normal space

Dataset

Feature	Type	Description
PassengerId	Object	Unique passenger identifier
HomePlanet	Categorical	Planet of origin
CryoSleep	Boolean	Whether the passenger opted for cryosleep
Cabin	Object	Location encoded as Deck/Number/Side
Destination	Categorical	Destination of the passenger
Age	Numeric	Passenger age
VIP	Boolean	Whether the passenger is VIP
RoomService, FoodCourt, ShoppingMall, Spa, VRDeck	Numeric	Onboard spending categories
Transported	Boolean	Target label

Methods and Algorithms

The project used three main supervised learning algorithms.
Each model was trained using the same training/validation split:

```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Loaded Data: (sample of raw dataset that was entered, reading train.csv and test.csv and storing them as pandas DataFrames)

[3]:	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Name	Transported
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	0.0	0.0	0.0	0.0	Maham Ofracculy	False
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	9.0	25.0	549.0	44.0	Juanna Vines	True
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	3576.0	0.0	6715.0	49.0	Altark Susent	False
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	1283.0	371.0	3329.0	193.0	Solam Susent	False
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	70.0	151.0	565.0	2.0	Willy Santantines	True

Processed data

To process the data we did the following in order to prepare the data so that machine learning models can understand it:

- Split the Cabin column into Deck, Cabin Number, and Side
- Identified numeric and categorical columns
- Filled missing numeric values with the median
- Filled missing categorical values with the mode (most common value)
- Ensured consistent data types to avoid warnings
- Removed irrelevant columns (Name, Cabin, PassengerId)
- Split data into training and validation sets (80/20 split)

[9]:	HomePlanet	CryoSleep	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Deck	CabinNum	Side
3600	Earth	False	TRAPPIST-1e	0.0	False	0.0	0.0	0.0	0.0	0.0	G	630	S
1262	Earth	True	TRAPPIST-1e	17.0	False	0.0	0.0	0.0	0.0	0.0	G	201	S
8612	Earth	False	PSO J318.5-22	35.0	False	0.0	0.0	0.0	0.0	0.0	G	1483	S
5075	Europa	True	55 Cancri e	26.0	False	0.0	0.0	0.0	0.0	0.0	D	164	S
4758	Earth	False	TRAPPIST-1e	13.0	False	0.0	0.0	60.0	1.0	5147.0	G	818	P

Logistic Regression Model

Using this model we:

- Created a **preprocessing pipeline** that:
 - Scales numeric values
 - One-hot encoding categorical values
- Fit a **Logistic Regression classifier**
- Made predictions on the validation set
- Calculated **Accuracy**, **ROC Curve**, and **AUC score**

Results:

Logistic Regression Accuracy: 0.7918

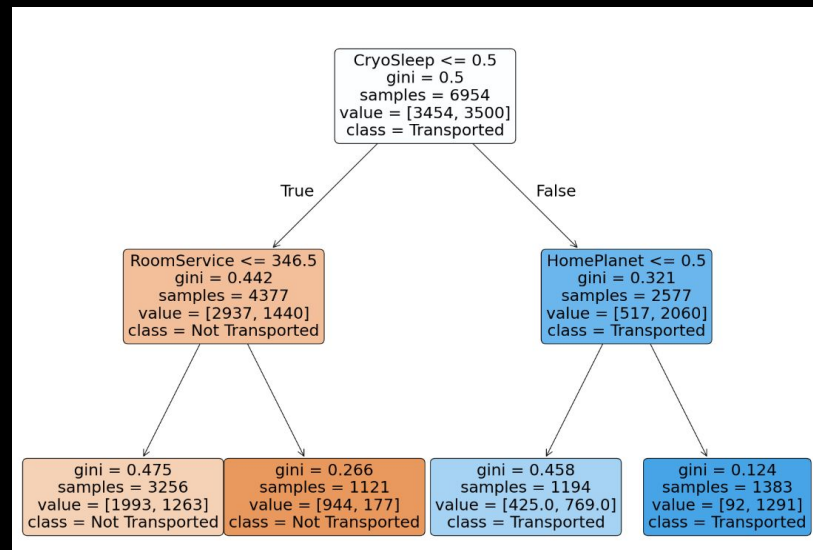
Logistic Regression AUC: 0.8815

```
preprocess_lr = ColumnTransformer(  
    transformers=[  
        ("num", StandardScaler(), numeric_features),  
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),  
         categorical_features)  
    ],  
    remainder="drop"  
)  
  
log_reg_model = Pipeline(steps=[  
    ("prep", preprocess_lr),  
    ("model", LogisticRegression(max_iter=500))  
)  
  
log_reg_model.fit(X_train, y_train)  
  
pred_lr = log_reg_model.predict(X_val)  
prob_lr = log_reg_model.predict_proba(X_val)[:, 1]  
  
acc_lr = accuracy_score(y_val, pred_lr)  
fpr_lr, tpr_lr, _ = roc_curve(y_val, prob_lr)  
auc_lr = auc(fpr_lr, tpr_lr)  
  
print("Logistic Regression Accuracy:", round(acc_lr, 4))  
print("Logistic Regression AUC:", round(auc_lr, 4))
```

Decision Tree Model

Goal - Build and evaluate a Decision Tree Classifier to predict whether a passenger was transported or not.

- The decision tree was one of the models used for this project because it is easy to interpret, fast at training, and can work well with multiple different feature types such as boolean, numeric, categorical, object.
- We made sure to convert boolean values into strings and Label Encoded all categorical columns.
- Train/Test split (80/20)



Decision Tree Results

DecisionTreeClassifier(random_state=42)

- Accuracy of Decision Tree model: 0.7343300747556066. (Estimate = .73)
- Area under the curve (AUC) Score: 0.7341585114516943. (Estimate = .73)
- Receiver-operating characteristic curve (ROC):

```
y_prob = dt.predict_proba(X_test)[:,-1]

falsepr, truepr, thresholds = roc_curve(y_test, y_prob)

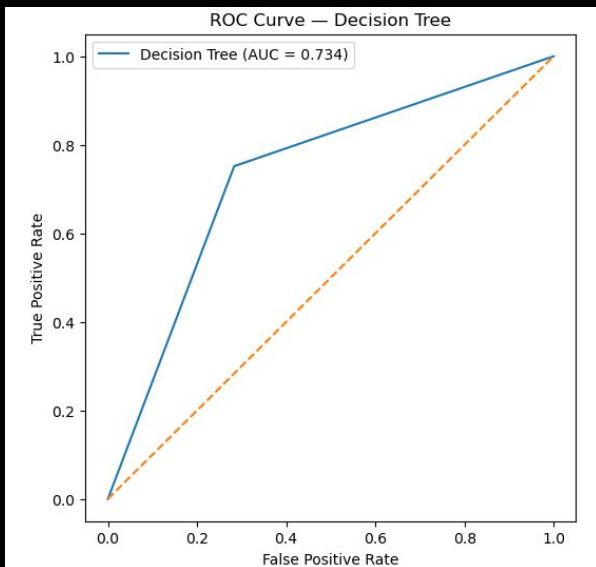
roc_auc = auc(falsepr, truepr)
print("AUC:", roc_auc)

plt.figure(figsize=(6,6))
plt.plot(falsepr, truepr, label=f"Decision Tree (AUC = {roc_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree")
plt.legend()
plt.show()
```

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
```

```
y_pred = dt.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Decision Tree Accuracy:", accuracy)
```



Random Forest Model

Using this model we:

- Trained a Random Forest Classifier using the same encoded dataset
- Combined many decision trees to improve accuracy
- Made predictions and computed Accuracy, ROC, and AUC

Results:

Random Forest Accuracy: 0.8085

Random Forest AUC: 0.8931

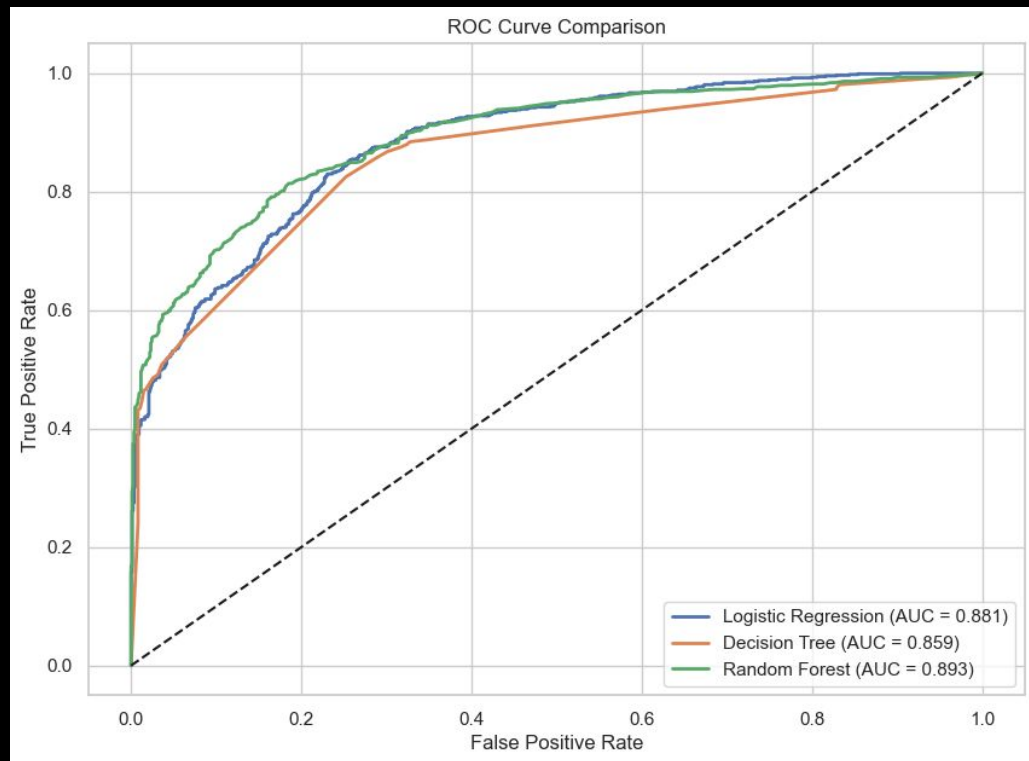
```
rf_model = RandomForestClassifier(  
    n_estimators=300,  
    max_depth=None,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    random_state=42  
)  
  
rf_model.fit(X_train_dt, y_train_dt)  
  
pred_rf = rf_model.predict(X_val_dt)  
prob_rf = rf_model.predict_proba(X_val_dt)[:, 1]  
  
acc_rf = accuracy_score(y_val_dt, pred_rf)  
fpr_rf, tpr_rf, _ = roc_curve(y_val_dt, prob_rf)  
auc_rf = auc(fpr_rf, tpr_rf)  
  
print("Random Forest Accuracy:", round(acc_rf, 4))  
print("Random Forest AUC:", round(auc_rf, 4))
```



MODEL PERFORMANCE SUMMARY

Logistic Regression:	Accuracy=0.792, AUC=0.881
Decision Tree:	Accuracy=0.784, AUC=0.859
Random Forest:	Accuracy=0.809, AUC=0.893

ROC Curve Comparison



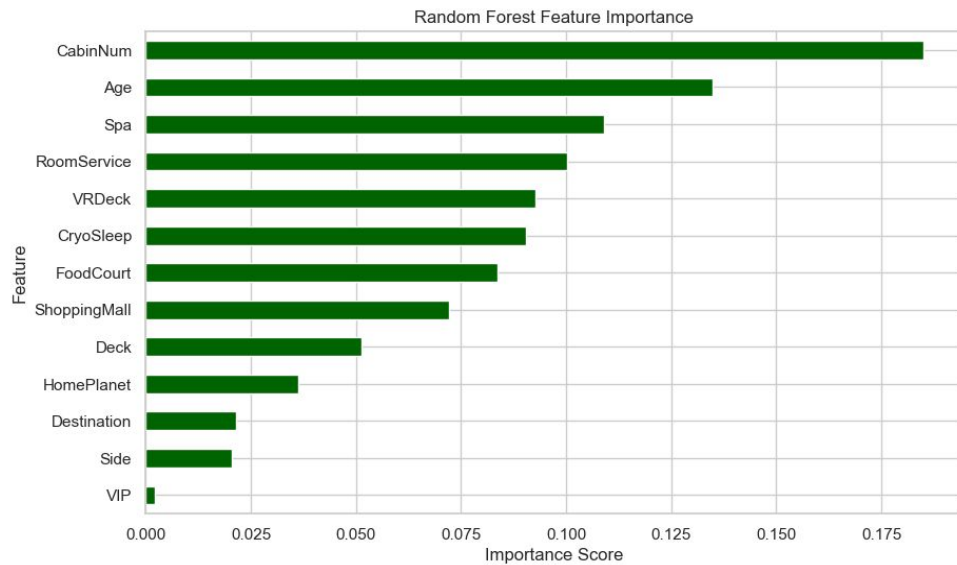
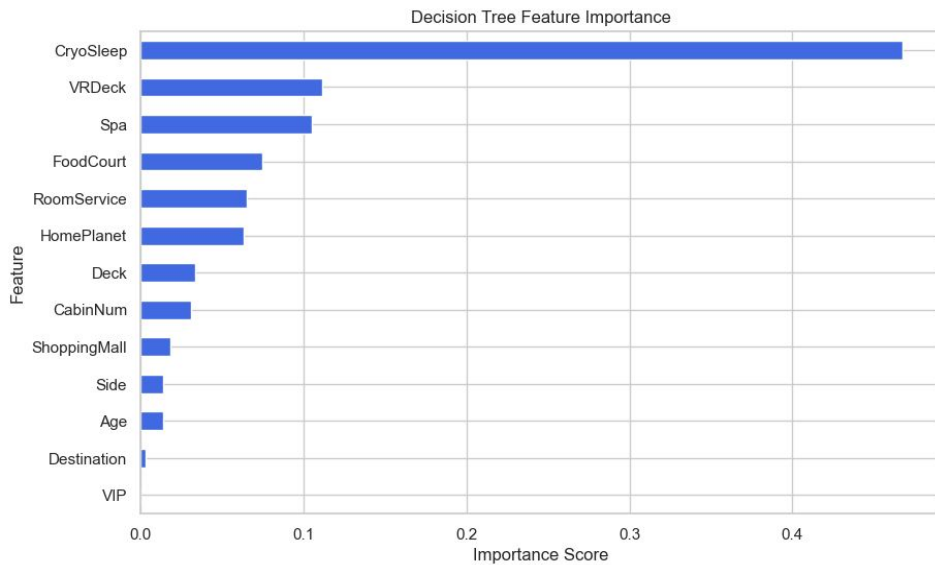
CROSS-VALIDATION SCORES

Logistic Regression CV Accuracy: 0.7872

Decision Tree CV Accuracy: 0.759

Random Forest CV Accuracy: 0.7851

Feature Importance Plots





Model impressions

Logistic Regression

Accuracy=0.79
AUC=0.881

- ❑ Excellent baseline
- ❑ Indicates your preprocessing (scaling + encoding) worked well
- ❑ Logistic regression often performs surprisingly well here
- ❑ AUC 0.881 means the model separates classes effectively

Decision Tree

Accuracy=0.78
AUC=0.859

- ❑ Slightly lower accuracy, which is normal
- ❑ Trees overfit easier unless tuned heavily
- ❑ AUC 0.859 is good for a single-tree model
- ❑ Confirms that non-linear splits help, but not as much as you need

Random Forest

Accuracy=0.8
AUC=0.893

- ❑ Best accuracy
- ❑ Best AUC
- ❑ Combines many trees → reduces overfitting

Results

All transported passengers
transported_passengers

	PassengerId	Transported
0	0013_01	True
2	0019_01	True
3	0021_01	True
5	0027_01	True
6	0029_01	True
...	...	...
4267	9260_01	True
4269	9263_01	True
4270	9265_01	True
4274	9271_01	True
4275	9273_01	True

2024 rows × 2 columns

Counting how many passengers were transported
num_transported = transported_passengers.shape[0]
num_total = submission_df.shape[0]

print(f"Transported Passengers: {num_transported} out of {num_total}")

Transported Passengers: 2024 out of 4277

Transported Passengers: 2024 out of 4277

Thank you